

Software Testing Assignment

Module - 6 (Core Java)

1) W.A.J.P to Take three numbers from the user and print the greatest number.

Program:

```
import java.util.Scanner;

public class GreatestNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int num1 = sc.nextInt();

        System.out.print("Enter second number: ");
        int num2 = sc.nextInt();

        System.out.print("Enter third number: ");
        int num3 = sc.nextInt();

        int greatest;

        if (num1 >= num2 && num1 >= num3) {
            greatest = num1;
        } else if (num2 >= num1 && num2 >= num3) {
            greatest = num2;
        } else {
            greatest = num3;
        }

        System.out.println("The greatest number is: " + greatest);
    }
}
```

Output:

```
Enter first number: 45
Enter second number: 78
Enter third number: 23
The greatest number is: 78
```

2) W.A.J.P in Java to display the first 10 natural numbers using while loop.

Program:

```
public class NaturalNumbers {
    public static void main(String[] args) {
        int i = 1;

        System.out.println("First 10 Natural Numbers:");
```

```
        while (i <= 10) {
            System.out.println(i);
            i++;
        }
    }
}
```

Output:

```
First 10 Natural Numbers:
1
2
3
4
5
6
7
8
9
10
```

3) W.A.J.P to find factorial for Given Number.**Program:**

```
import java.util.Scanner;

public class Factorial {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        long factorial = 1;

        for (int i = 1; i <= num; i++) {
            factorial = factorial * i;
        }

        System.out.println("Factorial of " + num + " is: " + factorial);
    }
}
```

Output:

```
Enter a number: 5
Factorial of 5 is: 120
```

4) W.A.J.P to check given number is Prime or not?

Program:

```
import java.util.Scanner;

public class PrimeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        boolean isPrime = true;

        if (num <= 1) {
            isPrime = false;
        } else {
            for (int i = 2; i <= num / 2; i++) {
                if (num % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }

        if (isPrime) {
            System.out.println(num + " is a Prime number.");
        } else {
            System.out.println(num + " is NOT a Prime number.");
        }
    }
}
```

Output:

```
Enter a number: 17
17 is a Prime number.

Enter a number: 12
12 is NOT a Prime number.
```

5) W.A.J.P to check given number is Armstrong or not?

Program:

```
import java.util.Scanner;

public class ArmstrongCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        int original = num;
        int digits = String.valueOf(num).length();
        int sum = 0;

        while (num > 0) {
            int digit = num % 10;
            sum = sum + (int) Math.pow(digit, digits);
            num = num / 10;
        }

        if (sum == original) {
            System.out.println(original + " is an Armstrong number.");
        } else {
            System.out.println(original + " is NOT an Armstrong number.");
        }
    }
}

```

Output:

```

Enter a number: 153
153 is an Armstrong number.

Enter a number: 123
123 is NOT an Armstrong number.

```

6) W.A.J.P for create Fibonacci Series.

Program:

```

import java.util.Scanner;

public class FibonacciSeries {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("How many terms do you want? ");
        int terms = sc.nextInt();

        int first = 0, second = 1;

        System.out.println("Fibonacci Series:");

        for (int i = 1; i <= terms; i++) {
            System.out.print(first + " ");
            int next = first + second;
            first = second;
            second = next;
        }
        System.out.println();
    }
}

```

```
    }  
}
```

Output:

```
How many terms do you want? 8  
Fibonacci Series:  
0 1 1 2 3 5 8 13
```

7) W.A.J.P to Print Pattern 1: 1 12 123 1234 12345

Program:

```
public class Pattern1 {  
    public static void main(String[] args) {  
        int rows = 5;  
  
        for (int i = 1; i <= rows; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print(j);  
            }  
            System.out.println();  
        }  
    }  
}
```

Output:

```
1  
12  
123  
1234  
12345
```

8) W.A.J.P to Print Pattern 2 (Right Aligned): 1 12 123 1234 12345

Program:

```
public class Pattern2 {  
    public static void main(String[] args) {  
        int rows = 5;  
  
        for (int i = 1; i <= rows; i++) {  
            // Print leading spaces  
            for (int s = rows; s > i; s--) {  
                System.out.print(" ");  
            }  
            // Print numbers  
            for (int j = 1; j <= i; j++) {  
                System.out.print(j);  
            }  
            System.out.println();  
        }  
    }  
}
```

Output:

```
1
12
123
1234
12345
```

9) W.A.J.P to Print Pattern 3: 1 01 101 0101 10101 010101**Program:**

```
public class Pattern3 {
    public static void main(String[] args) {
        int rows = 6;

        for (int i = 1; i <= rows; i++) {
            int start = (i % 2 == 1) ? 1 : 0;

            for (int j = 1; j <= i; j++) {
                System.out.print(start);
                start = (start == 1) ? 0 : 1;
            }
            System.out.println();
        }
    }
}
```

Output:

```
1
01
101
0101
10101
010101
```

10) W.A.J.P to Print Pattern 4: 1 2 2 3 3 3 4 4 4 4**Program:**

```
public class Pattern4 {
    public static void main(String[] args) {
        int rows = 4;

        for (int i = 1; i <= rows; i++) {
            // Print spaces for right alignment
            for (int s = rows; s > i; s--) {
                System.out.print(" ");
            }
            // Print number i, i times
            for (int j = 1; j <= i; j++) {
                System.out.print(i + " ");
            }
        }
    }
}
```

```

        System.out.println();
    }
}

```

Output:

```

    1
   2 2
  3 3 3
 4 4 4 4

```

11) W.A.J.P to Print Pattern 5 (Diamond Star Pattern): * *** ***** *** *

Program:

```

public class Pattern5 {
    public static void main(String[] args) {
        int rows = 3;

        // Upper half
        for (int i = 1; i <= rows; i++) {
            for (int s = rows; s > i; s--) {
                System.out.print(" ");
            }
            for (int j = 1; j <= (2 * i - 1); j++) {
                System.out.print("*");
            }
            System.out.println();
        }

        // Lower half
        for (int i = rows - 1; i >= 1; i--) {
            for (int s = rows; s > i; s--) {
                System.out.print(" ");
            }
            for (int j = 1; j <= (2 * i - 1); j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}

```

Output:

```

    *
   ***
  *****
   ***
    *

```

12) WAP to compute the sum of the first 100 prime numbers.

Program:

```
public class SumOf100Primes {

    static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        int count = 0;
        int num = 2;
        long sum = 0;

        while (count < 100) {
            if (isPrime(num)) {
                sum = sum + num;
                count++;
            }
            num++;
        }

        System.out.println("Sum of first 100 prime numbers is: " + sum);
    }
}
```

Output:

```
Sum of first 100 prime numbers is: 24133
```

13) WAP to sum values of an array.

Program:

```
public class ArraySum {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        int sum = 0;

        for (int i = 0; i < numbers.length; i++) {
            sum = sum + numbers[i];
        }

        System.out.print("Array elements: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
    }
}
```



```

    }
    System.out.println();
    System.out.println("Sum of array elements: " + sum);
}
}

```

Output:

```

Array elements: 10 20 30 40 50
Sum of array elements: 150

```

14) WAP to calculate the average value of array elements.

Program:

```

public class ArrayAverage {
    public static void main(String[] args) {
        double[] numbers = {15.0, 25.0, 35.0, 45.0, 55.0};

        double sum = 0;

        for (int i = 0; i < numbers.length; i++) {
            sum = sum + numbers[i];
        }

        double average = sum / numbers.length;

        System.out.print("Array elements: ");
        for (double num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
        System.out.println("Average of array elements: " + average);
    }
}

```

Output:

```

Array elements: 15.0 25.0 35.0 45.0 55.0
Average of array elements: 35.0

```

15) WAP to find the index of an array element.

Program:

```

import java.util.Scanner;

public class FindIndex {
    public static void main(String[] args) {
        int[] numbers = {10, 25, 37, 42, 58, 63};

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter element to search: ");
    }
}

```

```

        int target = sc.nextInt();

        int foundIndex = -1;

        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] == target) {
                foundIndex = i;
                break;
            }
        }

        if (foundIndex != -1) {
            System.out.println("Element " + target + " found at index: " +
foundIndex);
        } else {
            System.out.println("Element " + target + " not found in
array.");
        }
    }
}

```

Output:

```

Enter element to search: 37
Element 37 found at index: 2

Enter element to search: 99
Element 99 not found in array.

```

16) WAP to find the maximum and minimum value of an array.

Program:

```

public class MaxMinArray {
    public static void main(String[] args) {
        int[] numbers = {34, 7, 89, 12, 56, 3, 45};

        int max = numbers[0];
        int min = numbers[0];

        for (int i = 1; i < numbers.length; i++) {
            if (numbers[i] > max) {
                max = numbers[i];
            }
            if (numbers[i] < min) {
                min = numbers[i];
            }
        }

        System.out.print("Array: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}

```

```

        System.out.println("Maximum value: " + max);
        System.out.println("Minimum value: " + min);
    }
}

```

Output:

```

Array: 34 7 89 12 56 3 45
Maximum value: 89
Minimum value: 3

```

17) WAP to Compare Two Strings.

Program:

```

import java.util.Scanner;

public class CompareStrings {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first string: ");
        String str1 = sc.nextLine();

        System.out.print("Enter second string: ");
        String str2 = sc.nextLine();

        // Case-sensitive comparison
        if (str1.equals(str2)) {
            System.out.println("Both strings are EQUAL.");
        } else {
            System.out.println("Strings are NOT equal.");
        }

        // Case-insensitive comparison
        if (str1.equalsIgnoreCase(str2)) {
            System.out.println("Strings are equal (ignoring case).");
        } else {
            System.out.println("Strings are NOT equal even ignoring
case.");
        }

        // compareTo() comparison
        int result = str1.compareTo(str2);
        if (result == 0) {
            System.out.println("compareTo: Both strings are same.");
        } else if (result < 0) {
            System.out.println("compareTo: " + str1 + " comes before " +
str2 + "");
        } else {
            System.out.println("compareTo: " + str1 + " comes after " +
str2 + "");
        }
    }
}

```

```
}
```

Output:

```
Enter first string: Hello
Enter second string: hello
Strings are NOT equal.
Strings are equal (ignoring case).
compareTo: "Hello" comes before "hello"
```

18) WAP to concatenate a given string to the end of another string.**Program:**

```
import java.util.Scanner;

public class StringConcatenation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first string: ");
        String str1 = sc.nextLine();

        System.out.print("Enter second string: ");
        String str2 = sc.nextLine();

        // Method 1: Using + operator
        String result1 = str1 + str2;
        System.out.println("Using + operator: " + result1);

        // Method 2: Using concat() method
        String result2 = str1.concat(str2);
        System.out.println("Using concat(): " + result2);

        // Method 3: Using StringBuilder
        StringBuilder sb = new StringBuilder(str1);
        sb.append(str2);
        System.out.println("Using StringBuilder: " + sb.toString());
    }
}
```

Output:

```
Enter first string: Hello
Enter second string: World!
Using + operator: Hello World!
Using concat(): Hello World!
Using StringBuilder: Hello World!
```

19) WAP to demonstrate try catch block.

Program:

```
public class TryCatchDemo {
    public static void main(String[] args) {
        System.out.println("Program started.");

        try {
            int[] arr = new int[5];
            System.out.println("Accessing index 10 of array of size
5...");
            arr[10] = 50; // This causes ArrayIndexOutOfBoundsException
            System.out.println("This line will not execute.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception caught: " + e.getMessage());
            System.out.println("Array index is out of valid range!");
        }

        System.out.println("Program continues after exception handling.");
    }
}
```

Output:

```
Program started.
Accessing index 10 of array of size 5...
Exception caught: Index 10 out of bounds for length 5
Array index is out of valid range!
Program continues after exception handling.
```

20) WAP to demonstrate multiple catch blocks.

Program:

```
import java.util.Scanner;

public class MultipleCatchDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter a number: ");
            int num = sc.nextInt();

            int[] arr = new int[5];
            arr[num] = 100; // May throw ArrayIndexOutOfBoundsException

            int result = 100 / num; // May throw ArithmeticException
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
```

```

        System.out.println("Arithmetic Error: Cannot divide by
zero!");
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Array Error: Index is out of range!");
    } catch (Exception e) {
        System.out.println("General Exception: " + e.getMessage());
    } finally {
        System.out.println("Finally block always executes.");
    }
}
}

```

Output:

```

Enter a number: 0
Arithmetic Error: Cannot divide by zero!
Finally block always executes.

Enter a number: 8
Array Error: Index is out of range!
Finally block always executes.

```

21) WAP to create one thread by implementing Runnable interface in Class.

Program:

```

class MyRunnable implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable Thread: Count = " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted.");
            }
        }
    }
}

public class RunnableThreadDemo {
    public static void main(String[] args) {
        System.out.println("Main thread started.");

        MyRunnable myRunnable = new MyRunnable();
        Thread thread = new Thread(myRunnable);

        thread.start();

        System.out.println("Main thread finished.");
    }
}

```

Output:

```
Main thread started.  
Main thread finished.  
Runnable Thread: Count = 1  
Runnable Thread: Count = 2  
Runnable Thread: Count = 3  
Runnable Thread: Count = 4  
Runnable Thread: Count = 5
```

22) WAP to create one thread by extending Thread class in another Class.**Program:**

```
class MyThread extends Thread {  
    @Override  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Extended Thread: Count = " + i);  
            try {  
                Thread.sleep(400);  
            } catch (InterruptedException e) {  
                System.out.println("Thread was interrupted.");  
            }  
        }  
    }  
}  
  
public class ExtendThreadDemo {  
    public static void main(String[] args) {  
        System.out.println("Main thread started.");  
  
        MyThread myThread = new MyThread();  
        myThread.setName("MyCustomThread");  
        myThread.start();  
  
        System.out.println("Thread Name: " + myThread.getName());  
    }  
}
```

Output:

```
Main thread started.  
Thread Name: MyCustomThread  
Extended Thread: Count = 1  
Extended Thread: Count = 2  
Extended Thread: Count = 3  
Extended Thread: Count = 4  
Extended Thread: Count = 5
```

23) WAP to iterate through all elements in an ArrayList.

Program:

```
import java.util.ArrayList;
import java.util.Iterator;

public class ArrayListIterate {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");
        fruits.add("Orange");
        fruits.add("Grapes");

        // Method 1: for loop
        System.out.println("Method 1 - for loop:");
        for (int i = 0; i < fruits.size(); i++) {
            System.out.println(fruits.get(i));
        }

        // Method 2: for-each loop
        System.out.println("\nMethod 2 - for-each loop:");
        for (String fruit : fruits) {
            System.out.println(fruit);
        }

        // Method 3: Iterator
        System.out.println("\nMethod 3 - Iterator:");
        Iterator<String> it = fruits.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

Output:

```
Method 1 - for loop:
Apple
Banana
Mango
Orange
Grapes
```

```
Method 2 - for-each loop:
Apple
Banana
Mango
Orange
Grapes
```

```
Method 3 - Iterator:
```


Apple
Banana
Mango
Orange
Grapes

24) WAP to update specific array element by given element.

Program:

```
import java.util.Scanner;

public class UpdateArrayElement {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        System.out.print("Original array: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter index to update (0 to " + (numbers.length
- 1) + "): ");
        int index = sc.nextInt();

        System.out.print("Enter new value: ");
        int newValue = sc.nextInt();

        if (index >= 0 && index < numbers.length) {
            numbers[index] = newValue;
            System.out.println("Array updated successfully!");
        } else {
            System.out.println("Invalid index!");
        }

        System.out.print("Updated array: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}
```

Output:

```
Original array: 10 20 30 40 50
Enter index to update (0 to 4): 2
Enter new value: 99
Array updated successfully!
Updated array: 10 20 99 40 50
```

25) WAP to remove the third element from an ArrayList.

Program:

```
import java.util.ArrayList;

public class RemoveThirdElement {
    public static void main(String[] args) {
        ArrayList<String> cities = new ArrayList<>();

        cities.add("Mumbai");
        cities.add("Delhi");
        cities.add("Rajkot");    // 3rd element -> index 2
        cities.add("Surat");
        cities.add("Ahmedabad");

        System.out.println("Original ArrayList: " + cities);

        // Remove element at index 2 (third element)
        String removed = cities.remove(2);

        System.out.println("Removed element: " + removed);
        System.out.println("Updated ArrayList: " + cities);
    }
}
```

Output:

```
Original ArrayList: [Mumbai, Delhi, Rajkot, Surat, Ahmedabad]
Removed element: Rajkot
Updated ArrayList: [Mumbai, Delhi, Surat, Ahmedabad]
```

26) WAP to Copy one array into another.

Program:

```
import java.util.Arrays;

public class CopyArray {
    public static void main(String[] args) {
        int[] original = {5, 10, 15, 20, 25};

        // Method 1: Manual copy using loop
        int[] copy1 = new int[original.length];
        for (int i = 0; i < original.length; i++) {
            copy1[i] = original[i];
        }

        // Method 2: Using Arrays.copyOf()
        int[] copy2 = Arrays.copyOf(original, original.length);
    }
}
```

```

        // Method 3: Using System.arraycopy()
        int[] copy3 = new int[original.length];
        System.arraycopy(original, 0, copy3, 0, original.length);

        System.out.println("Original: " + Arrays.toString(original));
        System.out.println("Copy 1 (loop): " + Arrays.toString(copy1));
        System.out.println("Copy 2 (Arrays.copyOf): " +
Arrays.toString(copy2));
        System.out.println("Copy 3 (System.arraycopy): " +
Arrays.toString(copy3));
    }
}

```

Output:

```

Original: [5, 10, 15, 20, 25]
Copy 1 (loop): [5, 10, 15, 20, 25]
Copy 2 (Arrays.copyOf): [5, 10, 15, 20, 25]
Copy 3 (System.arraycopy): [5, 10, 15, 20, 25]

```

27) WAP to reverse an array of integer values.

Program:

```

import java.util.Arrays;

public class ReverseArray {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5, 6, 7};

        System.out.println("Original array: " + Arrays.toString(numbers));

        // Two pointer technique to reverse
        int left = 0;
        int right = numbers.length - 1;

        while (left < right) {
            int temp = numbers[left];
            numbers[left] = numbers[right];
            numbers[right] = temp;
            left++;
            right--;
        }

        System.out.println("Reversed array: " + Arrays.toString(numbers));
    }
}

```

Output:

```

Original array: [1, 2, 3, 4, 5, 6, 7]
Reversed array: [7, 6, 5, 4, 3, 2, 1]

```

28) WAP to find the second largest element in an array.

Program:

```
public class SecondLargest {
    public static void main(String[] args) {
        int[] numbers = {34, 7, 89, 12, 56, 89, 45};

        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] > largest) {
                secondLargest = largest;
                largest = numbers[i];
            } else if (numbers[i] > secondLargest && numbers[i] !=
largest) {
                secondLargest = numbers[i];
            }
        }

        System.out.print("Array: ");
        for (int num : numbers) System.out.print(num + " ");
        System.out.println();
        System.out.println("Largest: " + largest);
        System.out.println("Second Largest: " + secondLargest);
    }
}
```

Output:

```
Array: 34 7 89 12 56 89 45
Largest: 89
Second Largest: 56
```

29) W.A.J.P. Create an abstract class 'Parent' with a method 'message'. It has two subclasses each having a method with the same name 'message' that prints "This is first subclass" and "This is second subclass" respectively. Call the methods by creating an object for each subclass.

Program:

```
// Abstract parent class
abstract class Parent {
    abstract void message();
}

// First subclass
class FirstChild extends Parent {
    @Override
    public void message() {
        System.out.println("This is first subclass");
    }
}
```

```

// Second subclass
class SecondChild extends Parent {
    @Override
    public void message() {
        System.out.println("This is second subclass");
    }
}

public class AbstractClassDemo {
    public static void main(String[] args) {
        FirstChild obj1 = new FirstChild();
        SecondChild obj2 = new SecondChild();

        obj1.message();
        obj2.message();
    }
}

```

Output:

```

This is first subclass
This is second subclass

```

30) W.A.J.P. which will ask the user to enter his/her marks (out of 100). Define a method to display grades: 91-100=AA, 81-90=AB, 71-80=BB, 61-70=BC, 51-60=CD, 41-50=DD, <=40=Fail

Program:

```

import java.util.Scanner;

public class GradeCalculator {

    static String getGrade(int marks) {
        if (marks >= 91 && marks <= 100) {
            return "AA";
        } else if (marks >= 81 && marks <= 90) {
            return "AB";
        } else if (marks >= 71 && marks <= 80) {
            return "BB";
        } else if (marks >= 61 && marks <= 70) {
            return "BC";
        } else if (marks >= 51 && marks <= 60) {
            return "CD";
        } else if (marks >= 41 && marks <= 50) {
            return "DD";
        } else if (marks >= 0 && marks <= 40) {
            return "Fail";
        } else {
            return "Invalid marks!";
        }
    }
}

```

```

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your marks (out of 100): ");
        int marks = sc.nextInt();

        String grade = getGrade(marks);

        System.out.println("Marks: " + marks);
        System.out.println("Grade: " + grade);
    }
}

```

Output:

```

Enter your marks (out of 100): 85
Marks: 85
Grade: AB

Enter your marks (out of 100): 38
Marks: 38
Grade: Fail

```

31) W.A.J.P. to create a custom exception if Customer withdraw amount which is greater than account balance then program will show custom exception otherwise amount will deduct from account balance. [Account Balance: 2000, Withdraw Amount: 2500]

Program:

```

import java.util.Scanner;

// Custom Exception class
class InsufficientBalanceException extends Exception {
    private double shortAmount;

    public InsufficientBalanceException(double shortAmount) {
        super("Sorry, insufficient balance, you need more "
            + shortAmount + " Rs. To perform this transaction.");
        this.shortAmount = shortAmount;
    }
}

// Bank Account class
class BankAccount {
    private double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }

    public void withdraw(double amount) throws
        InsufficientBalanceException {
        if (amount > balance) {

```

```

        double shortage = amount - balance;
        throw new InsufficientBalanceException(shortage);
    } else {
        balance = balance - amount;
        System.out.println("Withdrawal successful!");
        System.out.println("Amount withdrawn: " + amount + " Rs.");
        System.out.println("Remaining balance: " + balance + " Rs.");
    }
}
}

public class CustomExceptionDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(2000);

        System.out.println("Account balance is: 2000");
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter withdraw amount: ");
        double withdrawAmount = sc.nextDouble();

        try {
            account.withdraw(withdrawAmount);
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

Output:

```

Account balance is: 2000
Enter withdraw amount: 2500
Sorry, insufficient balance, you need more 500.0 Rs. To perform this
transaction.

```